

EXP11: Map Coloring Problem (CSP)

Name: Y. Roshitha (192424400)

Aim:

To develop a Python program that solves the Map Coloring problem as a Constraint Satisfaction Problem (CSP), so that adjacent regions are never assigned the same color using backtracking search.

Algorithm:

Step 1: Represent each region as a variable and the available colors as the domain for each variable.

Step 2: Represent the adjacency relationship between regions as constraints (neighbouring regions must not share a color).

Step 3: If all regions are assigned a color, return the solution.

Step 4: Otherwise, select the next unassigned region.

Step 5: Try assigning each available color to the region and check whether it satisfies all neighbour constraints.

Step 6: If a valid color is found, assign it and move to the next region.

Step 7: If no color is valid, backtrack to the previous region and try a different color.

Step 8: Repeat Steps 3 to 7 until all regions are validly colored, then display the final assignment.

Source Code:

```
# Program for Map Coloring using Constraint Satisfaction Problem (CSP)
```

```
def is_valid(region, color, coloring, neighbours):
    for neighbour in neighbours[region]:
        if coloring.get(neighbour) == color:
            return False
    return True

def backtrack(regions, colors, coloring, neighbours):
    if len(coloring) == len(regions):
        return coloring

    region = [r for r in regions if r not in coloring][0]

    for color in colors:
        if is_valid(region, color, coloring, neighbours):
            coloring[region] = color
            result = backtrack(regions, colors, coloring, neighbours)
            if result:
                return result
            del coloring[region]
```

```
return None
```

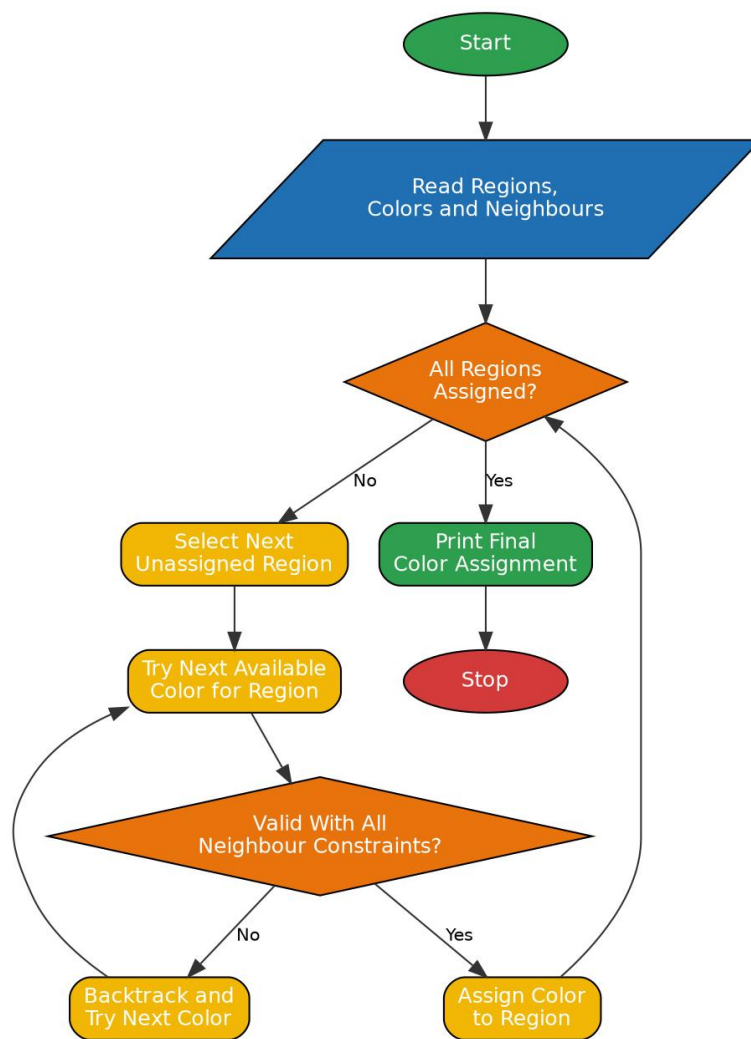
```
regions = ['WA', 'NT', 'SA', 'Q', 'NSW', 'V', 'T']  
colors = ['Red', 'Green', 'Blue']
```

```
neighbours = {  
    'WA': ['NT', 'SA'],  
    'NT': ['WA', 'SA', 'Q'],  
    'SA': ['WA', 'NT', 'Q', 'NSW', 'V'],  
    'Q': ['NT', 'SA', 'NSW'],  
    'NSW': ['SA', 'Q', 'V'],  
    'V': ['SA', 'NSW', 'T'],  
    'T': ['V']  
}
```

```
solution = backtrack(regions, colors, {}, neighbours)
```

```
print("Map Coloring Solution:")  
for region in regions:  
    print(f"{region} : {solution[region]}")
```

Flowchart:



Output:

```
>>> ===== RESTART: C:/Users/yanar/Documents/CSA1717/EXP11.py =====
Map Coloring Solution:
WA : Red
NT : Green
SA : Blue
Q : Red
NSW : Green
V : Red
T : Green
```

Result:

Thus the program for Map Coloring using Constraint Satisfaction Problem (CSP) was written, executed successfully and the output was verified.